POLITECNICO DI TORINO

Corso di Laurea in Cybersecurity

Tesi di Laurea

An incremental approach to automatic firewall configuration

Relatori
prof. Fulvio Valenza
prof. Daniele Bringhenti
prof. Francesco Pizzato

Candidato
Filippo Del Ministro

Anno accademico 2025-2026

Modern enterprise networks are complex, distributed systems in which security policy enforcement is a critical and continuous operational challenge. As topologies grow in scale and heterogeneity, the manual configuration of packet-filtering firewalls becomes increasingly error-prone. Misconfigurations can expose sensitive resources or degrade service availability. The shift toward microsegmentation, distributing enforcement across multiple firewall instances throughout the network, improves security granularity but amplifies the management effort. Automated tools that generate formally correct and optimal configurations are therefore essential.

VEREFOO (VErification REasoning on Firewall Optimization) addresses this need by automatically computing firewall allocation and configuration from a network topology and a set of Network Security Requirements (NSRs). It encodes the problem as a Maximum Satisfiability Modulo Theories (MaxSMT) instance solved by the Z3 solver, guaranteeing formal correctness and minimality of the result. However, VEREFOO follows a monolithic approach: every time the NSR set changes, the entire problem is re-encoded and solved from scratch, scaling poorly as the network or requirement set grows.

React-VEREFOO addresses this by taking an incremental approach. It identifies which requirements are new, removed, or unchanged, and restricts the solver to only the affected parts of the network. This reduces reconfiguration cost when few requirements change, but the benefit diminishes when a large fraction of the NSR set is modified simultaneously.

This thesis pushes the incremental principle to its extreme. The proposed Iterative React-VEREFOO introduces new NSRs one at a time: at each step, the current configuration is used as the starting state and exactly one requirement is added. React-VEREFOO computes the minimal update, and the result becomes the input to the next iteration. By construction, the reconsidered portion of the network is always minimal, keeping each MaxSMT subproblem small regardless of the total number of requirements.

Index

Capitolo 1

Introduction

Modern enterprise networks are large, dynamic infrastructures that change continuously. New services are deployed, nodes are added or removed, and connectivity requirements evolve at a pace that security policy management must match. At the same time, these networks span hundreds of heterogeneous nodes distributed across sites and organisational boundaries, making security enforcement a complex and ongoing operational challenge [1]. Packet-filtering firewalls are the primary mechanism for enforcing security policies, and their correct configuration is essential to prevent unauthorized access, data breaches, and service disruptions. A thorough discussion of this context is provided in Chapter 2.

The traditional approach to firewall configuration — manual translation of high-level security requirements into device-level rules — does not scale to modern network environments. As networks grow in size and change more frequently, the volume and complexity of rules to be managed quickly exceeds human capacity. More critically, manual processes offer no formal guarantee of correctness: a single misconfiguration may silently violate security requirements or block legitimate traffic, with consequences that can remain undetected for extended periods [2].

VEREFOO was developed to address this problem through full automation. Given a network topology and a set of Network Security Requirements (NSRs), it automatically determines where firewalls should be placed and what filtering rules each should enforce, with formal guarantees of correctness and optimality [3]. It achieves this by encoding the problem as a Maximum Satisfiability Modulo Theories (MaxSMT) instance solved by the Z3 solver. However, VEREFOO follows a monolithic strategy: any change to the NSR set triggers a complete recomputation from scratch, making it poorly suited to dynamic environments where requirements change frequently.

React-VEREFOO addressed this limitation by adopting an incremental approach [4]. Rather than discarding the existing configuration, it identifies which NSRs are new, which have been removed, and which are unchanged, and restricts the solver to only the affected parts of the network. This yields significant speedups when the number of modified requirements is small. However, when a large fraction of the NSR set changes simultaneously, the advantage diminishes and performance converges toward that of the monolithic baseline.

This thesis proposes a strategy that preserves the benefits of the incremental approach regardless of how many new requirements are introduced at once. The key idea is to push the incremental principle to its extreme: new NSRs are introduced one at a time, each in a separate invocation of React-VEREFOO that takes the current configuration as its starting point. By construction, the modified fraction at each step is always minimal — a single requirement — so the subproblem remains small and the performance advantage of the incremental approach is consistently in effect. The full formulation of the approach and the thesis objectives are presented in Chapters 3 and 4.

The remainder of the thesis is organised as follows. Chapter 2 provides the necessary background on network security automation, VEREFOO, and React-VEREFOO. Chapter 3 states the thesis objectives and introduces the iterative approach at a high level. Chapter 4 develops the approach from a theoretical perspective. Chapter 5 describes the implementation and reports the experimental evaluation on three benchmark topologies: the Stanford university network, the

CESNET academic network, and the Texas 2000 synthetic topology. Chapter 6 draws conclusions and outlines directions for future work.

Capitolo 2

Background

2.1 Context

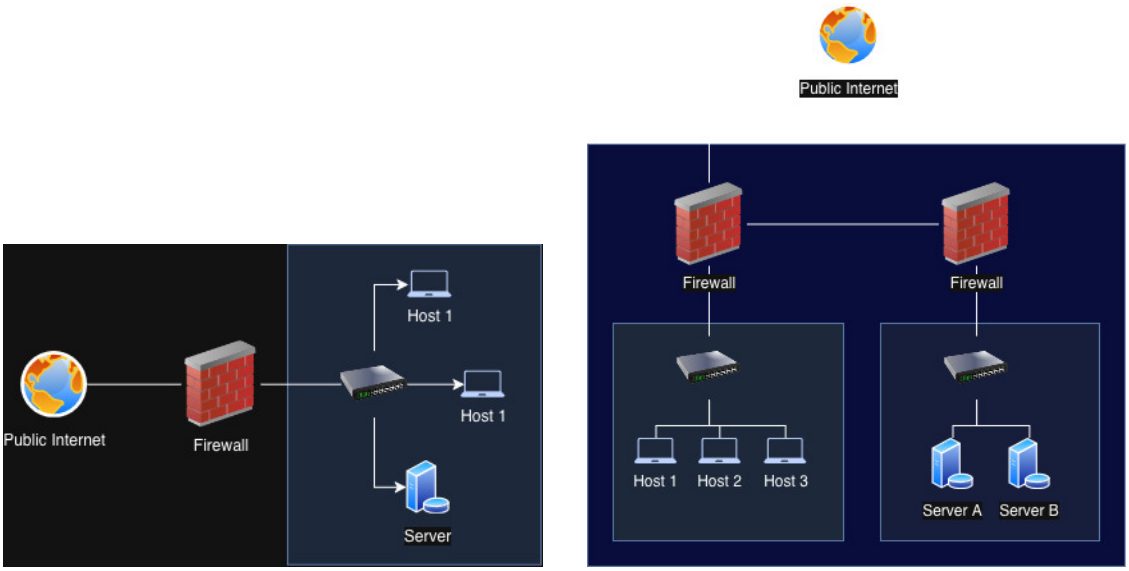
Network security is a critical concern for virtually every organization, from small businesses to large enterprises and public institutions. All of them rely on computer networks to support their operations, and all of them are exposed to the risk of unauthorized access, data breaches, and service disruptions. As a result, enforcing a well-defined and correct security policy is not optional, but instead is a fundamental operational requirement.

Modern enterprise networks are characterized by growing complexity: hundreds of interconnected nodes, heterogeneous services, and security policies that must be consistently enforced across distributed infrastructure. Moreover, network topologies are not static: new services are deployed, nodes are added or removed, and connectivity requirements evolve continuously, making security management an ongoing and demanding task. The cost of managing network security in this context is significant, as it requires specialized expertise, advanced planning, and constant monitoring [1].

A central challenge in this landscape stems from the evolution of network security architectures. Traditionally, a single perimeter firewall was sufficient to protect a network by controlling traffic at its boundary. However, as networks grew in scale and complexity, this model proved inadequate: a single point of control cannot enforce fine-grained policies across a heterogeneous internal topology. Modern networks have therefore evolved toward microsegmentation, deploying multiple packet-filtering firewall instances strategically distributed across the topology to enforce security policies at a much more granular level [5]. This distributed approach offers clear advantages such as enabling finer-grained traffic control, reduces single points of failure, and allows security policies to be enforced closer to the traffic sources, but it also introduces considerable management complexity. Each firewall instance must be individually configured with a set of filtering rules, and those rules must collectively enforce the intended security policy across the entire network, without conflicts or gaps.

In this context, the configuration of firewalls becomes a critical task. On one hand, overly permissive rules may allow unauthorized traffic to reach sensitive resources, exposing the network to attacks. On the other hand, overly restrictive rules may block legitimate traffic, degrading service availability and causing operational disruptions. Both failure modes carry significant consequences, and yet both are common outcomes of manual configuration process which is inherently prone to errors especially for large-scale networks [2].

The gap between the complexity of modern networks and the limitations of manual security management calls for automated approaches that can generate correct and optimal firewall configurations without human intervention. It is in this context that VEREFOO was developed: a tool for the automated, formally correct allocation and configuration of distributed firewalls, designed to bridge exactly this gap.



(a) Traditional perimeter firewall architecture.

(b) Distributed firewall architecture (microsegmentation).

Figura 2.1: Evolution of network security architectures: from a single perimeter firewall to distributed packet-filtering instances deployed across the topology.

2.2 VEREFOO

2.2.1 Overview

VEREFOO (VErification REasoning on Firewall Optimization) is a tool designed to automate the allocation and configuration of packet-filtering firewalls in a network, eliminating the need for manual intervention entirely [3]. Given a network topology and a set of Network Security Requirements (NSRs) — high-level specifications describing which traffic flows must be allowed or denied between network endpoints [6] — VEREFOO automatically determines where firewalls should be placed within the topology and generates the corresponding filtering rules for each allocated instance. The resulting configuration is expressed in a vendor-independent format, which can then be translated into concrete rules for specific implementations such as iptables, BPF, or OVS.

The tool provides three formal guarantees. *Automation* ensures that no human intervention is required at any stage of the process. *Formal correctness* guarantees that the generated configuration satisfies all specified NSRs by construction, eliminating the risk of undetected misconfigurations. *Optimality* ensures that both the number of allocated firewall instances and the number of generated rules are minimised, reducing resource consumption and improving performance. These properties are achieved by encoding the firewall allocation and configuration problem as a partial weighted Maximum Satisfiability Modulo Theories (MaxSMT) instance, which is then solved using the Z3 SMT solver. The hard constraints of the problem encode the correctness requirements, while the soft constraints encode the optimality objectives; the solver finds a solution that satisfies all hard constraints and maximises the sum of satisfied soft constraint weights.

2.2.2 Network Security Requirements

Network Security Requirements (NSRs) are the high-level specification of the security policy that a network must enforce. Rather than describing firewall rules directly, NSRs express what the administrator intends: which traffic flows between pairs of endpoints must be permitted and which must be denied. This level of abstraction decouples the intent of the security policy from its low-level implementation, allowing VEREFOO to take full responsibility for translating requirements into concrete filtering rules [3].

Each NSR refers to a specific traffic flow, identified by five attributes: source address, destination address, transport protocol, source port, and destination port. The administrator specifies, for each such flow, whether it must be reachable (i.e., allowed end-to-end through the network) or isolated (i.e., blocked). VEREFOO supports four different policy modes that determine how the requirements are interpreted: *whitelisting*, where all traffic is blocked by default and only explicitly allowed flows are permitted; *blacklisting*, where all traffic is allowed by default and only explicitly listed flows are denied; and two mixed modes (*rule-oriented specific* and *security-oriented specific*), which accept both reachability and isolation requirements without a fixed default behavior, differing in how unspecified flows are handled [3].

NSRs are provided to VEREFOO as part of its input, alongside the network topology. Internally, they are represented and stored as a structured set of logical constraints, expressed in a vendor-independent intermediate language. A higher-level specification language is also supported, from which the corresponding medium-level NSRs are automatically derived through a translation step. VEREFOO processes this set together with the Allocation Graph derived from the topology, and encodes each NSR as a hard constraint in the MaxSMT problem: the solver is required to find a firewall allocation and rule set that satisfies all of them without exception. NSRs that are satisfied by the same firewall rules are grouped and handled jointly to minimise the total number of generated rules, while each unresolvable conflict or missing coverage results in a non-enforceability report to the administrator [3, 6].

2.2.3 Service Graph

A Service Graph (SG) is the logical representation of a virtual network: a directed graph in which nodes represent Network Functions (NFs) such as web servers, load balancers, caches, or client subnetworks while edges represent the connectivity between them. The SG captures all possible end-to-end traffic paths through the network at an abstract level, without exposing low-level forwarding infrastructure such as switches and routers. It is defined taking into consideration solely the services that the network must provide, independently of any security consideration [3].

The SG serves as the structural input to VEREFOO: it defines the space of possible firewall positions and the traffic paths that any allocated firewall must be able to intercept. To make this explicit, VEREFOO automatically derives an internal representation called the *Allocation Graph* (AG) from the SG. For each link between any pair of nodes in the SG, the AG introduces a placeholder element called an *Allocation Place* (AP) which represents a candidate position where a firewall instance may be allocated. Those Allocation Places are then used by VEREFOO during the optimal placement of firewalls. Optionally, Verefoo allow to manually constrain this process by forcing a firewall into a specific AP or excluding certain APs from consideration, for example to model pre-existing hardware firewalls that cannot be moved [3].

VEREFOO uses the Allocation Graph (AG) jointly with the Network Security Requirements (NSRs) to formulate the MaxSMT problem. Each AP corresponds to boolean decision variable that the solver uses to determines the optimal firewall placement and what filtering rules each firewall should enforce, such that all NSRs are satisfied and the total number of allocated firewalls and rules is minimised.

2.2.4 VEREFOO Output

When the MaxSMT problem is solved successfully, VEREFOO produces two complementary outputs [3]. The first is the *firewall allocation scheme*: an updated version of the Allocation Graph in which each selected Allocation Place hosts an active firewall instance, with the minimum number of firewalls necessary to enforce all NSRs. The second is the *Filtering Policy* (FP) for each allocated firewall: a complete, auto-generated rule set that defines how the firewall must handle every traffic flow passing through it. Each FP is expressed in a vendor-independent abstract language, making it independent of any specific firewall implementation. It consists of a default action (either drop-all in whitelisting mode, or allow-all in blacklisting mode) and a minimal set of explicit rules covering the traffic flows specified by the NSRs. Both the default action and the explicit rules are guaranteed to be anomaly-free, with no conflicts or redundancies among them.

Together, these two outputs constitute the full security configuration of the network at the logical level of the Service Graph, independent of the physical infrastructure. A subsequent translation step converts the abstract filtering policies into concrete rules for specific implementations such as iptables, BPF, or OVS [3]. If instead no solution exists, for example because user-defined constraints on the AG have excluded all viable firewall positions, VEREFOO returns a non-enforceability report in place of the configuration.

2.2.5 Verefoo validation

VEREFOO has been validated on both synthetic and real-world network topologies, including well-known benchmarks such as the Stanford and CESNET networks, demonstrating reliability and scalability across a range of network sizes and requirement sets [3].

Despite these strengths, VEREFOO's monolithic approach has a fundamental scalability limitation: the entire set of NSRs and the full network topology are encoded into a single MaxSMT problem. As the number of requirements or the size of the network grows, the complexity of the problem increases significantly, leading to prohibitive computation times. This limitation motivates the development of more efficient alternatives, starting with React-VEREFOO.

Further details on the experimental validation of VEREFOO and its comparison with the iterative approach proposed in this thesis can be found in Chapter 5.

2.3 React-VEREFOO

In operational environments, network security policies are not static: NSRs evolve over time as new services are deployed, topology changes occur, or threat models are updated [7]. A fundamental limitation of the monolithic approach used by VEREFOO is that any change to the NSR set triggers a full recomputation of the firewall configuration from scratch. React-VEREFOO addresses this by offering a more efficient approach in the dynamic context described above, avoiding the need to discard a valid existing configuration when only a small portion of the requirements has changed [4].

React-VEREFOO takes two inputs: the existing firewall configuration (allocation scheme and filtering rules) and a pair of NSR sets — the *initial set* R_i, describing the requirements already enforced by the current configuration, and the *target set* R_t, describing the requirements that must be enforced after reconfiguration. The approach proceeds in three steps [4].

Step 1 — NSR classification. React-VEREFOO partitions every NSR in $R_i \cup R_t$ into one of three groups:

- $R_d = \{r \in R_i \mid r \notin R_t\}$: *deleted* requirements, previously enforced but no longer needed. They are simply excluded from the new problem formulation.
- $R_a = \{r \in R_t \mid r \notin R_i\}$: *added* requirements, new NSRs that must be enforced but are not yet satisfied. These are the sole driver of reconfiguration.
- $R_k = \{r \in R_t \mid r \in R_i\}$: *kept* requirements, already correctly enforced and still valid. The nodes satisfying them are left unchanged.

Step 2 — Identification of the network area to reconfigure. For each added requirement in R_a, React-VEREFOO analyses the Allocation Graph to determine which nodes are in conflict with the new requirement and therefore need to be reconsidered. The procedure differs depending on whether the added requirement is an isolation or a reachability constraint [4].

For an added *isolation* requirement (traffic that must be blocked), the algorithm examines all atomic flows associated with that requirement. If a flow is already blocked by an allocated firewall, no action is needed for that flow. If instead no node along the path currently blocks the traffic, all nodes on that path are flagged as eligible for reconfiguration, since one of them must be configured to enforce the isolation.

For an added *reachability* requirement (traffic that must be allowed end-to-end), the algorithm checks whether at least one atomic flow reaches the destination without being blocked. If such a flow already exists, the requirement is already satisfied and no reconfiguration is needed. Otherwise, the nodes that are currently blocking the traffic are flagged as eligible for reconfiguration.

In both cases, the algorithm selects all nodes that *could potentially* be used to fulfill the new requirement, since the optimal choice among them is left to the solver. Nodes not flagged by any added requirement retain their current configuration and are excluded from the decision variables of the subsequent MaxSMT problem.

Step 3 — MaxSMT problem formulation and solving. Once the reconfigurable area has been identified, React-VEREFOO formulates a MaxSMT problem. This problem models traffic flows for all NSRs in the target set, ensuring that the final configuration is correct with respect to all requirements. However, only the flagged nodes appear as free decision variables; the configuration of all other nodes is treated as fixed. This restriction dramatically reduces the size of the search space compared to a full recomputation [4].

The soft constraints are also adjusted to bias the solver toward reusing the existing configuration: allocating an already-deployed firewall is preferred over placing a new one, and retaining

an existing rule is preferred over generating a new one. This minimises the number of changes applied to the network, reducing the time needed to deploy the new configuration.

The result is an updated allocation scheme and a revised set of filtering policies that satisfy all requirements in the target set, with formal correctness guaranteed by construction. As with VEREFOO, the output may not be globally optimal, since only a subset of the network is reconsidered, but it is optimal within the identified reconfiguration area.

Experimental results confirm that React-VEREFOO is significantly more efficient than vanilla VEREFOO when the proportion of modified NSRs is small, as the reduced problem size leads to much shorter solving times. However, when a large fraction of the requirements changes, the benefit diminishes and performance converges toward the monolithic baseline. This observation motivates the approach investigated in this thesis: by pushing the incremental logic to its extreme and introducing only one new NSR per iteration. The proportion of modified requirements is always minimised, potentially achieving consistent speedups regardless of the total number of requirements.

Capitolo 3

Thesis Objectives

3.1 Idea of the Solution

To understand the motivation of this thesis, it is necessary to examine how React-VEREFOO determines what to reconfigure. Given an initial set of NSRs R_i already enforced in the network and a target set R_t that must be enforced after reconfiguration, React-VEREFOO classifies every requirement into one of three groups [4]:

- $R_d = \{r \in R_i \mid r \notin R_t\}$: *deleted* requirements, present in the initial configuration but no longer needed;
- $R_a = \{r \in R_t \mid r \notin R_i\}$: *added* requirements, new requirements that must be enforced but are not yet satisfied;
- $R_k = \{r \in R_t \mid r \in R_i\}$: *kept* requirements, already enforced and still valid in the target configuration.

Only the added requirements R_a drive the reconfiguration: React-VEREFOO identifies which nodes in the Allocation Graph are in conflict with each added NSR and restricts the solver to those areas, while leaving the rest of the configuration untouched. The final MaxSMT problem models all traffic flows for the target set $R_t = R_a \cup R_k$, but the decision variables are limited to the nodes identified as needing reconfiguration, making the problem significantly smaller than a full recomputation [4].

The key performance parameter is the proportion of kept requirements, which can be defined as:

$$\text{PercReqKept} = \frac{|R_k|}{|R_k| + |R_a| + |R_d|}$$

The higher this ratio, the smaller the added and deleted sets, and consequently the narrower the network area that must be reconsidered. Experimental results in [4] confirm that the approach yields its greatest speedups at high values of PercReqKept (90% and above), and that performance converges toward the vanilla baseline as PercReqKept decreases. This is consistent with real-world operational patterns: studies of large-scale infrastructure [4] report that the vast majority of configuration updates affect only a small fraction of the total rule set, making high PercReqKept the typical case in practice.

The key idea of this thesis is to push this observation to its logical extreme. Instead of processing all new NSRs in a single reconfiguration step, the proposed approach introduces them **one at a time**. At each step, only one new requirement is presented to React-VEREFOO as added, while everything enforced so far falls into the kept set. This means the reconfiguration area identified at each step is always the smallest it could possibly be — a single requirement drives each solver call, regardless of how many requirements must ultimately be incorporated. The proportion of kept

requirements is therefore always at its theoretical maximum for the given state of the network, and grows larger with every iteration as the kept set expands.

Formally, at each iteration i, the current configuration enforces $R_k^{(i)} = R_i \cup \{A_1, \ldots, A_{i-1}\}$ and the single new requirement A_i constitutes the entire added set $R_a^{(i)} = \{A_i\}$. This gives:

$$\text{PercReqKept}^{(i)} = \frac{|R_k^{(i)}|}{|R_k^{(i)}| + 1}$$

which grows toward 100% as more requirements are incorporated, and is always strictly higher than the PercReqKept achievable by processing the same requirements in a single batch. By construction, each individual solver call thus operates on the smallest possible incremental problem.

Concretely, the process unfolds as a chain of reconfiguration steps. The network starts from an existing valid configuration. One new requirement is then selected and handed to React-VEREFOO, which updates the configuration to accommodate it. The result, the new allocation scheme and updated filtering rules becomes the starting point for the next step. Another requirement is then introduced, the process repeats, and so on until every new requirement has been incorporated. At no point is the full set of new requirements processed at once; the tool always sees exactly one addition on top of an otherwise stable configuration.

Formally, let P_0 denote the initial set of requirements already enforced in the network and $\{A_1, A_2, A_3, \ldots\}$ the ordered sequence of new NSRs to be introduced. The chain of React-VEREFOO invocations is:

1. initial: P_0, additional: $\{A_1\}$
2. initial: $\{P_0, A_1\}$, additional: $\{A_2\}$
3. initial: $\{P_0, A_1, A_2\}$, additional: $\{A_3\}$
4. …

The total number of solver invocations equals the number of new NSRs — a linear cost — but each individual invocation is far cheaper than a single reconfiguration over the full batch, since the solver always operates considering one new requirement only. Whether the cumulative saving per step outweighs the cost of repeating the process n times is the central empirical question addressed in Chapter 5.

3.2 Expected Benefits and Trade-offs

The proposed approach is expected to yield consistent performance improvements over both vanilla VEREFOO and standard React-VEREFOO, at the cost of some trade-offs that are important to acknowledge.

Expected benefits. Because each invocation of React-VEREFOO sees a problem with exactly one new requirement, the solver is always operating on the smallest possible incremental problem. This is in contrast to standard React-VEREFOO, which can receive an arbitrarily large batch of new NSRs and whose efficiency degrades as the batch grows. The proposed iterative approach removes this dependency on batch size: regardless of how many NSRs must be processed in total, each individual solver call remains minimal.

Trade-offs. The main cost of the approach is the linear number of solver invocations: where vanilla VEREFOO solves one problem and React-VEREFOO solves one (possibly large) problem, the iterative variant solves n small problems. Each invocation carries a fixed overhead — parsing, constraint encoding, and solver initialization — which accumulates over the n iterations. Whether the per-step gain in solver efficiency outweighs this overhead is ultimately an empirical question, and one that the experimental evaluation in Chapter 5 is designed to answer.

A second trade-off concerns the quality of the final configuration. Vanilla VEREFOO optimises globally over the full NSR set, producing a configuration that minimises the total number of firewalls and rules jointly. The proposed iterative approach, by contrast, optimises locally at each step. In particular, a firewall placed during the first iteration to satisfy the first new NSR (defined as A_1 in previous example) may not be the best structural choice once the subsequent new requirements $(A_2, \ldots, A_n)$ are taken into account in the following iterartions. This potential loss of global optimality is a known limitation of any greedy sequential strategy.

The approach investigated in this thesis focuses exclusively on the addition of new NSRs. The case of NSR removal is not considered: when a requirement must be revoked, the iterative strategy described here does not apply, and a separate reconfiguration procedure would be needed. Similarly, the thesis considers only the whitelisting and blacklisting policy modes supported by React-VEREFOO, and the evaluation is conducted on three specific benchmark topologies. Generalisation to other policy modes or network types is left as future work.

Capitolo 4

The Iterative VEREFOO Approach

Capitolo 5

Implementation and Validation

5.1 Implementation

5.2 Testing

5.2.1 Test Presentation

5.2.2 Test Configurations

5.2.3 Results

Figure 5.1 reports the average execution times measured across the three benchmark topologies. Each data point represents the mean over 20 runs. The Y-axis uses a logarithmic scale to accommodate the wide range of values observed, particularly for larger networks.

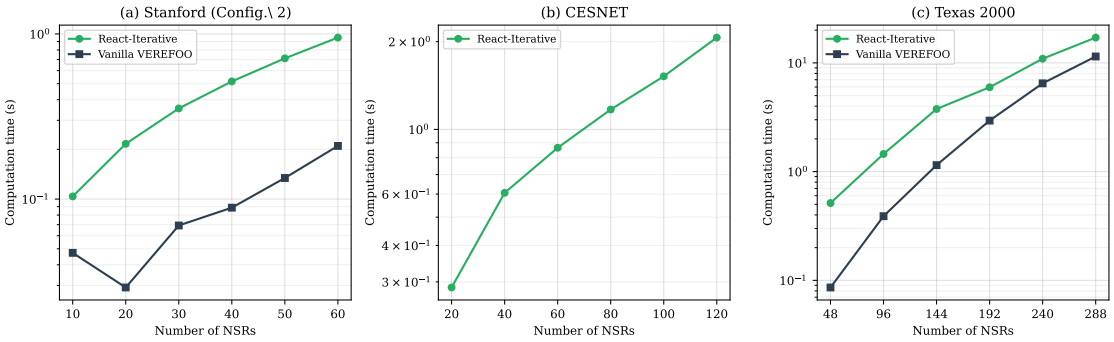


Figura 5.1: Execution time comparison between React-Iterative VEREFOO and Vanilla VEREFOO across three benchmark topologies.

5.2.4 Comparison and Discussion

Capitolo 6

Conclusion and Future Work

Bibliografia

[1] D. Bringhenti, G. Marchetto, R. Sisto, and F. Valenza, "Automation for Network Security Configuration: State of the Art and Research Trends," *ACM Computing Surveys*, vol. 56, no. 3, 2023.

[2] D. Bringhenti, S. Bussa, R. Sisto, and F. Valenza, "Atomizing Firewall Policies for Anomaly Analysis and Resolution," *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 3, 2025.

[3] D. Bringhenti, G. Marchetto, R. Sisto, F. Valenza, and J. Yusupov, "Automated Firewall Configuration in Virtual Networks," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, pp. 1559–1576, March-April 2023.

[4] F. Pizzato, D. Bringhenti, R. Sisto, and F. Valenza, "Automatic and Optimized Firewall Reconfiguration," in *Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS 2024)*, 2024.

[5] D. Bringhenti, R. Sisto, and F. Valenza, "A Novel Abstraction for Security Configuration in Virtual Networks," *Computer Networks*, vol. 228, p. 109745, 2023.

[6] D. Bringhenti, S. Bussa, R. Sisto, and F. Valenza, "A Two-Fold Traffic Flow Model for Network Security Management," *IEEE Transactions on Network and Service Management*, vol. 21, no. 4, 2024.

[7] D. Bringhenti, R. Sisto, and F. Valenza, "Autonomous Attack Mitigation Through Firewall Reconfiguration," *International Journal of Network Management*, vol. 34, no. 1, 2024.